UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1

Manual de Usuario - Practica 2

Mauricio Joel Gómez Barrios - 202031478

11 de abril de 2026

Índice.

Introducción..... 3
Requisitos del sistema..... 4
Cómo ejecutar la aplicación:..... 4
Interfaz..... 4
Sección de Gramática Wison..... 5
Sección de Resultados..... 6
Cómo definir una gramática en Wison..... 7
Evaluación de cadenas y árbol de derivación..... 8
Visualización de errores..... 9

Introducción.

Es un compilador web que permite definir gramáticas libres de contexto usando el lenguaje Wison, verificar que sean válidas para un analizador LL(1) y evaluar cadenas de entrada generando el árbol de derivación correspondiente junto con los conjuntos First, Follow y la Tabla Predictiva TAS.

Requisitos del sistema.

- **Sistema Operativo:** Linux, Windows o macOS.
- **Navegador:** Google Chrome, Firefox o Edge.
- **Node.js:** v20 o superior.

Cómo ejecutar la aplicación:

1) Abrir una terminal en la carpeta /backend y ejecutar npm install para instalar las dependencias en el backend.
2) Ejecuta node server.js, el servidor se quedará escuchando en el puerto 3000.
3) Abrir otra terminal en la carpeta /frontend y ejecutar npm install para instalar las dependencias en el frontend.
4) Ejecuta ng serve -o, la aplicación estará disponible en http://localhost:4200 y se abrirá la pestaña automáticamente.

Interfaz.

- **Área de Gramática Wison - Editor:** Un espacio de texto donde se puede escribir la gramática usando el lenguaje Wison, aceptando comentarios de línea (#) y de bloque (/** */).
- **Campo de Texto a Evaluar:** Un espacio de texto de una sola línea donde se ingresa la cadena de entrada que se desea analizar con la gramática definida.
- **Botón "Compilar y Generar Tablas":** Envía la gramática y la cadena al servidor para su análisis. El botón se deshabilita mientras la petición está en proceso y muestra el texto "Analizando...".
- **Panel de Errores:** Aparece debajo del formulario únicamente cuando el servidor detecta errores léxicos, sintácticos o semánticos. Muestra la lista detallada de todos los problemas encontrados.
- **Panel de Resultados:** Visible sólo cuando el análisis es exitoso. Contiene las tablas de First/Follow, la tabla predictiva TAS y el árbol de derivación visual.

Sección de Gramática Wison.

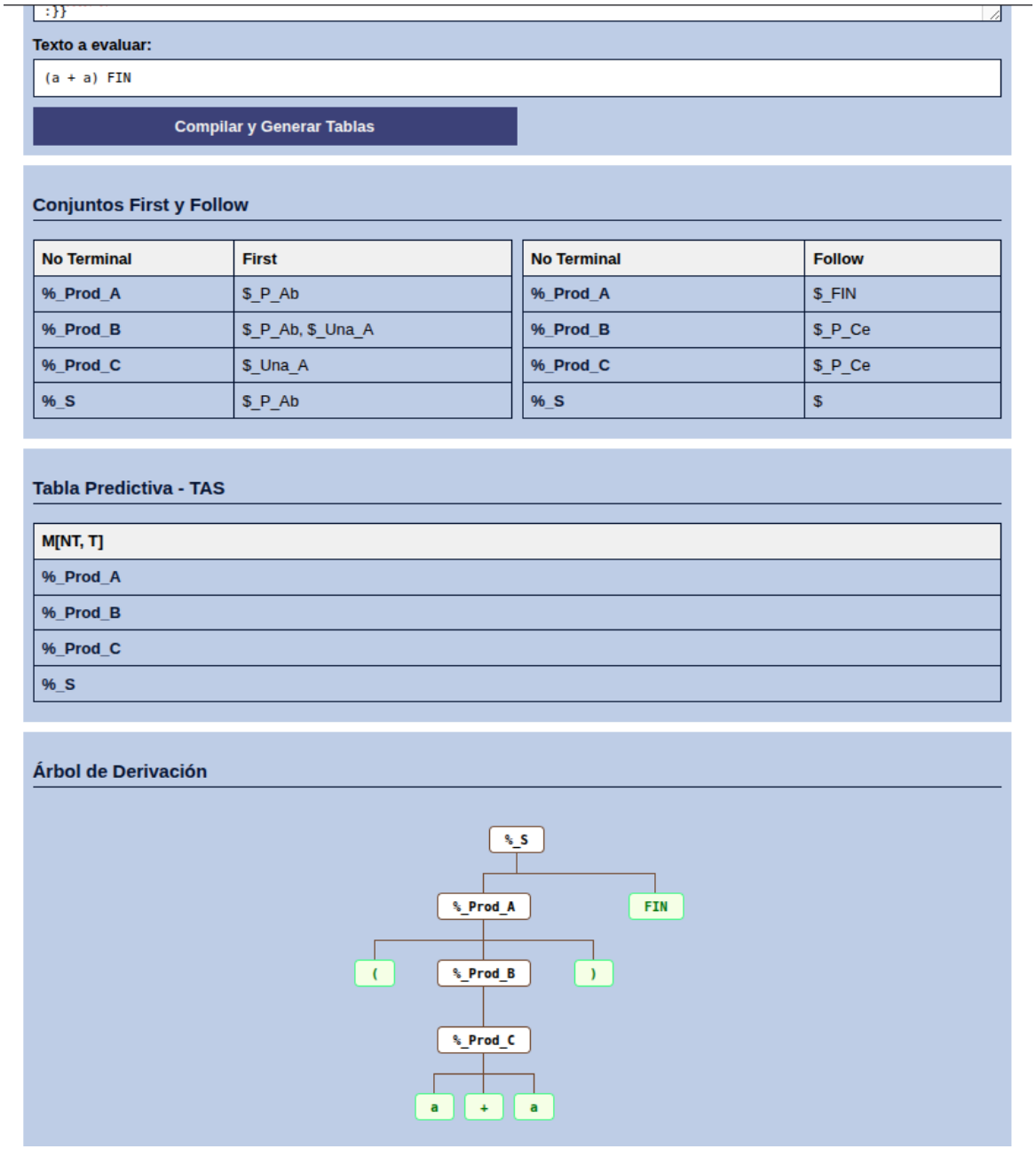
El área principal de la interfaz contiene el editor de gramática. En él se puede escribir directamente la definición de tu gramática en el lenguaje Wison, o pegar el contenido de un archivo previamente preparado.

- **Área de texto de la gramática:** Acepta comentarios de línea (#) y comentarios de bloque (/** ... */).
- **Campo de cadena a evaluar:** Debajo del área de gramática está el espacio de texto de una sola línea. Aquí se debe ingresar la cadena de tokens separados por espacios que se desea probar con la gramática definida.

Sección de Resultados.

Al presionar el botón compilar, la interfaz muestra los resultados organizados en secciones:

- **Conjuntos First y Follow:** Se muestran dos tablas. La tabla izquierda muestra el conjunto First de cada no terminal y la tabla derecha muestra el conjunto Follow correspondiente.
- **Tabla Predictiva TAS:** Muestra la Tabla de Análisis Sintáctico con los no terminales como filas y los terminales como columnas. Cada celda contiene la producción que debe aplicarse.
- **Árbol de Derivación:** Visualización gráfica del árbol generado al analizar la cadena de entrada. Los nodos intermedios (no terminales) se muestran con borde marrón y fondo blanco; los nodos hoja (terminales) se muestran con borde y texto en color verde simulando un arbolito.



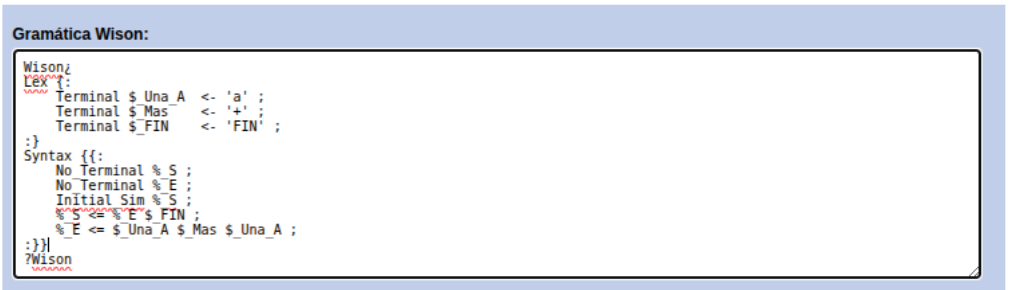
Cómo definir una gramática en Wison.

Para crear una gramática válida en el lenguaje Wison estan los pasos a continuación:

1) Iniciar la estructura escribiendo la palabra reservada Wison¿ al comienzo.
2) Definir los terminales dentro del bloque Lex {: ... :}. Cada terminal se declara así:
   Terminal $_NOMBRE <- EXPRESION_REGULAR ;
   Las expresiones regulares válidas son:
   - **Cadenas literales:** entre comillas simples, por ejemplo 'a', '+', 'FIN'.
   - **Rango de letras:** [aA-zZ] para reconocer cualquier letra del alfabeto.
   - **Rango de dígitos:** [0-9] para reconocer cualquier dígito del 0 al 9.
   - **Operador Kleene (*):** cero o más repeticiones, por ejemplo [0-9]*.
   - **Cerradura positiva (+):** una o más repeticiones, por ejemplo [0-9]+.
   - **Opcional (?):** cero o una vez, por ejemplo [0-9]?.
   - **Concatenación:** agrupando expresiones con paréntesis, por ejemplo ([0-9]*)($_Punto)([0-9]+).
   - **Referencia a terminal:** Se puede referenciar un terminal ya declarado usando su nombre $_NOMBRE.
3) Define los no terminales y el símbolo inicial dentro del bloque Syntax {{: ... :}}:
   - Declara cada no terminal: No_Terminal %_Nombre ;
   - Especifica el símbolo inicial: Initial_Sim %_Nombre ;
   - Escribe las producciones: %_Cabeza <= cuerpo1 | cuerpo2 ;
4) Se cierra la estructura escribiendo ?Wison al final.

Ejemplo completo de gramática Wison:

```
Wison¿
Lex {:
        Terminal $_Una_A  <- 'a' ;
        Terminal $_Mas    <- '+' ;
        Terminal $_FIN    <- 'FIN' ;
:}
Syntax {{:
        No_Terminal %_S ;
        No_Terminal %_E ;
        Initial_Sim %_S ;
        %_S <= %_E $_FIN ;
        %_E <= $_Una_A $_Mas $_Una_A ;
:}}
?Wison
```

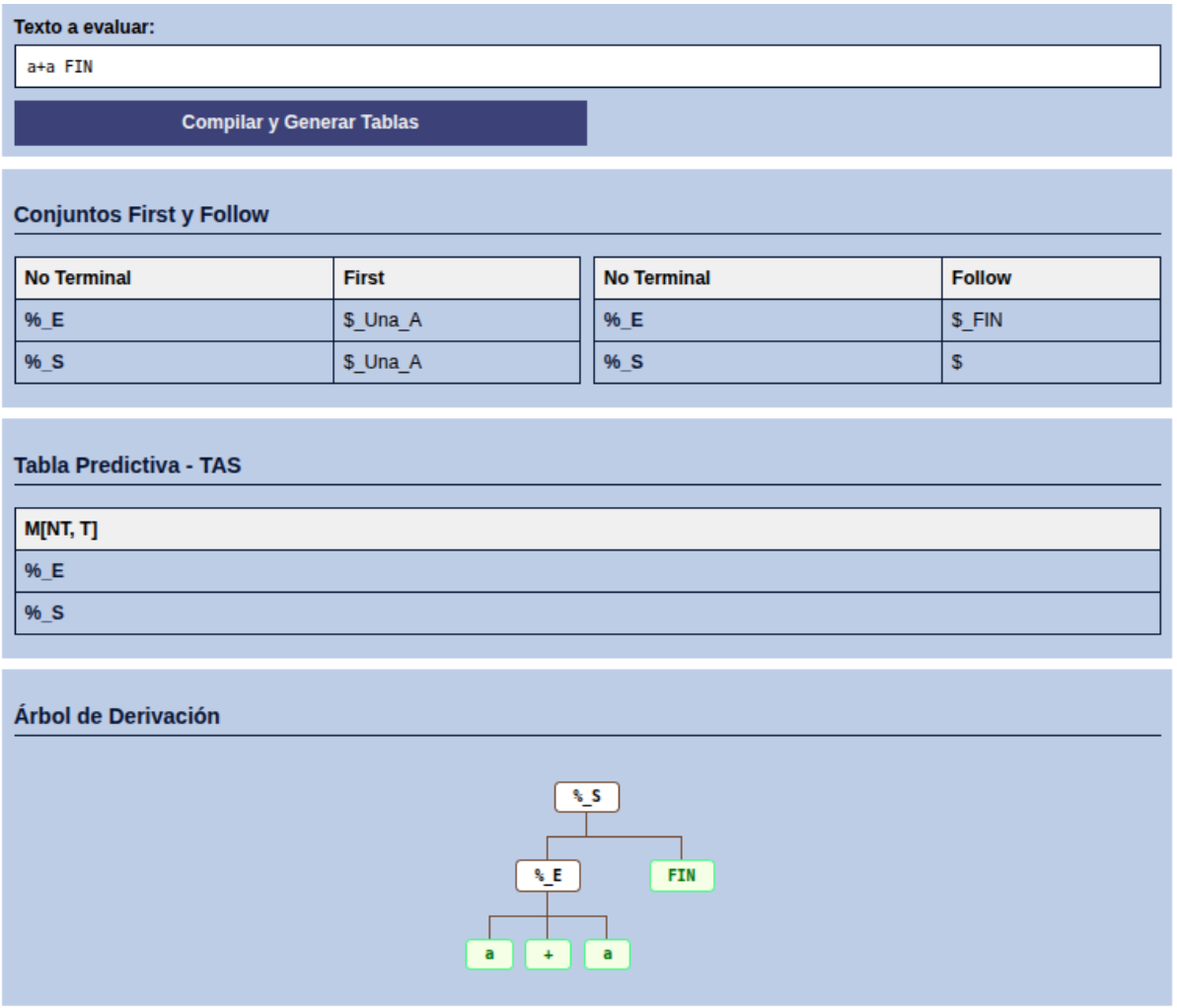


Evaluación de cadenas y árbol de derivación.

Para evaluar si una cadena es aceptada por la gramática definida:

1) Escribe la gramática Wison en el área de texto superior.
2) Ingresa la cadena de prueba en el campo de texto inferior.
3) Presionar el botón "Compilar y Generar Tablas".
4) Si la gramática y la cadena son correctas, la aplicación mostrará:
   - Los conjuntos First y Follow calculados para cada no terminal.
   - La Tabla Predictiva (TAS) con todas las entradas M[NT, T] generadas.
   - El árbol de derivación construido durante el análisis LL(1) con pila.

- **Lectura del árbol de derivación:**
  - Nodos con borde café y fondo blanco: son no terminales (nodos internos del árbol).
  - Nodos con borde verde y fondo crema: son terminales (hojas del árbol).
  - Nodo ε: indica que el no terminal derivó en la cadena vacía (producción epsilon).



Visualización de errores.
Si se cometió un error al escribir la gramática (error léxico, sintáctico) o la gramática no es apta para LL(1) (recursividad por la izquierda, falta de factorización, colisiones en la TAS), o si la cadena de prueba contiene tokens no reconocidos o no es aceptada, el panel de errores se activará debajo del formulario mostrando la lista completa de mensajes de error para que se pueda identificar y corregir el problema.

- **Tipos de errores que pueden reportarse:**
  - **Error léxico:** carácter no reconocido en la gramática Wison o en la cadena de prueba.
  - **Error sintáctico:** estructura incorrecta de la gramática (bloque Lex o Syntax mal formados).
  - **Error semántico:** terminal o no terminal usado sin haber sido declarado, símbolo inicial faltante, o declaraciones duplicadas.
  - **Error LL:** recursividad por la izquierda detectada, falta de factorización o colisión en la TAS (la gramática no es LL(1)).
  - **Error de cadena:** la cadena de prueba no es aceptada por la gramática definida.